



Projet 1 LINFO1341 - SRTP

Groupe 27: Lai Loïc, Zhang Chi

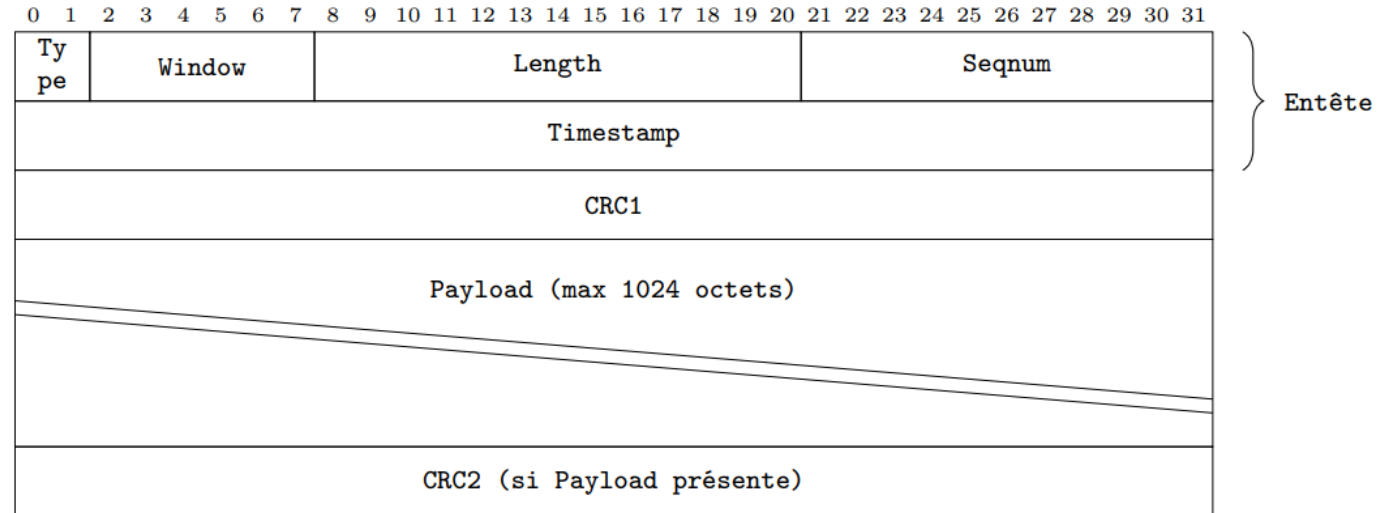
Architecture Générale

packet.py

Contient une classe Packet, les objets de cette classe contiennent les informations des packets (Type, window, timestamp, payload).

Cette classe a deux méthodes qui permet de créer un packet et de débiller un packet:

```
def pack(self):  
    """  
    Créer un packet STPR  
    :return: Packet STPR en bytes  
    """  
  
    @staticmethod  
    def unpack(packet):  
        """  
        Unpack un packet STPR  
        :param data: Les bytes du packet  
        :return: Un objet Packet ou None si invalide  
        """
```



Format: Network Byte Order (Big-Endian)

Gestion du timestamp

```
# Découpage du contenu du fichier en packet de taille 1024 max
while True:
    chunk = f.read(1024)
    packets.append(Packet( type: 1, window, seqnum, int(time.time()), chunk))

    if not chunk:
        packet = Packet( type: 1, window, seqnum, int(time.time()), payload=b'')
        sock.sendto(packet.pack(), client_address)
        break
```

Dans timestamp on a mis la valeur que donne time.time().

Utilité:

- Il permet de tracker le temps de création des packets. Il sera utile lorsqu'on veut savoir si un paquet ou un ACK met trop de temps à se transmettre et définir une limite de temps pour les paquet qui ne vont jamais arriver (timeout).
- Il permet d'identifier les paquets qui sont trop vieux en cas de forte congestion
- Il permet d'estimer le temps de voyage d'un packets entre le serveur et le client (RTT). Le serveur calcule la différence de temps pour ajuster son comportement.

Serveur

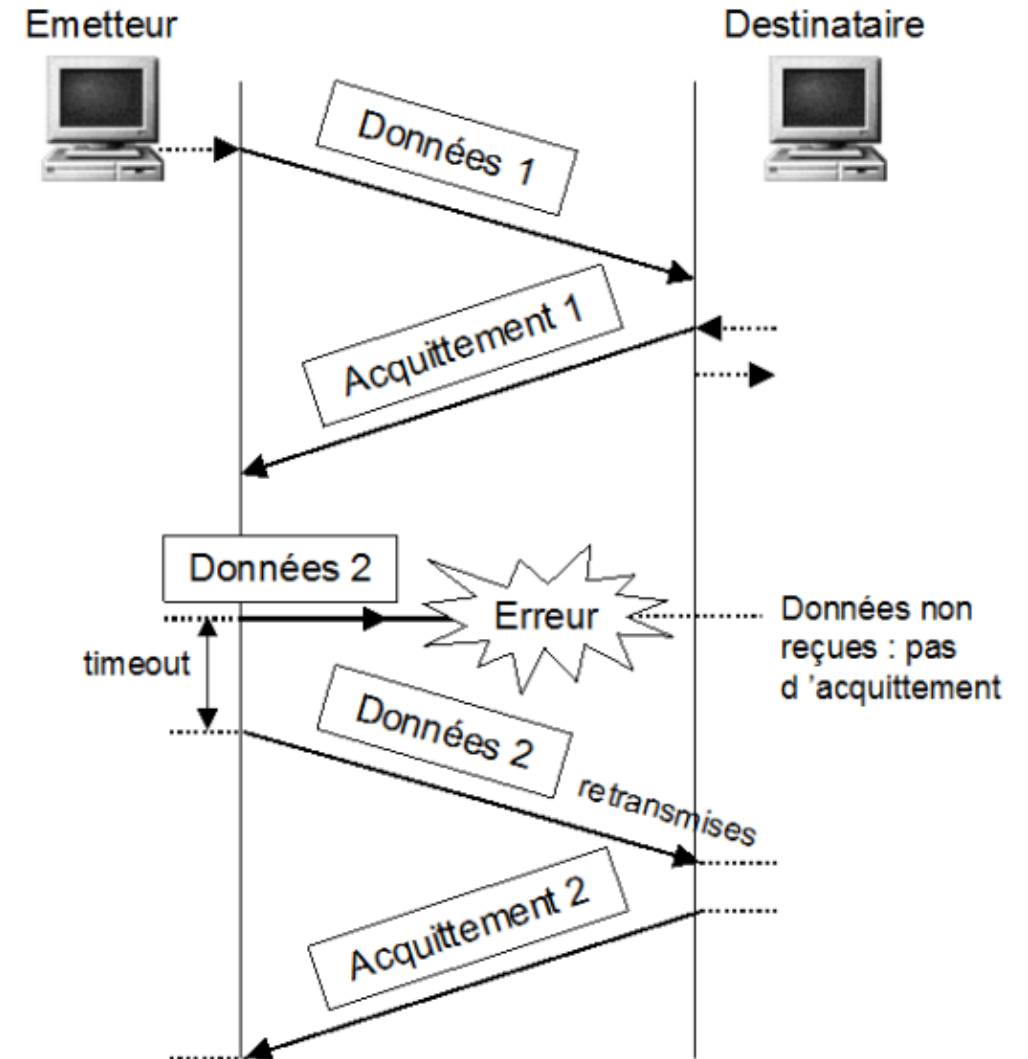
Découpage dynamique : Il lit le fichier par blocs de **1024 octets** maximum pour créer les paquets.

Gestion des états : Il garde en mémoire quel paquet a été envoyé, lequel a été reçu (ACK) et lequel doit être renvoyé.

La Fenêtre Glissante : Le serveur peut gérer de **1 à 63 paquets** en même temps, défini par la valeur du window dans les paquets.

Gestion du seqnum : Le serveur peut gérer les fichiers de grande taille (seqnum > 2047)

Surveillance du temps : Pour chaque paquet envoyé, le serveur surveille le temps écoulé. (Timeout)



Valeur du retransmission timeout

On a choisi le temps du timeout à **0,5 seconde**.

Retransmission automatique : Si après **0,5 seconde** un paquet n'a toujours pas reçu son ACK, le serveur considère qu'il a été perdu et le renvoie immédiatement.

Indépendance : Chaque paquet de la fenêtre possède son propre suivi temporel dans un dictionnaire. (timestamp)

Selective Repeat : renvoie que le paquet manquant.

Ce n'est pas la valeur optimale.

```
# Retransmission du message due au timeout
now = time.time()
for i in range(last_ack + 1, last_sent + 1):
    if i in timestamp and (now - timestamp[i] > 0.5):
        print(f"Timeout, retransmission du seq {packets[i].seqnum}", file=sys.
            sock.sendto(packets[i].pack(), client_address)
            timestamp[i] = now
```

Client



Gestion du Désordre : Le client utilise un dictionnaire (buffer) pour stocker les paquets qui arrivent avant ceux attendus.

Dès que le paquet manquant arrive, le client le retire son buffer.

Filtrage des doublons : Le client ignore les paquets déjà reçus ou ceux dont le numéro de séquence est inférieur à celui attendu

Partie critique

1. Le Retransmission Timeout (RTO) Fixe

En cas de réseau instable, il y aura un délai de 0,5s. Lorsqu'un paquet est perdu, la fenêtre se remplit rapidement, puis le serveur s'arrête totalement de transmettre pendant 0,5s.

2. La Taille de la Fenêtre

La fenêtre de 63 ne sera pas suffisante pour des gros fichiers

3. Selective repeat

Plus efficace que le Go-Back-N, renvoie que les packets manquant.

Test

```
tests > test_client.py > ...
1 import os
2 import sys
3 import tempfile
4 import pytest
5 import socket
6 sys.path.append("src")
7 import client
8 from packet import Packet
9
10
11 #faux socket pour test
12 class FakeSocket:
13     def __init__(self, rep):
14         self.rep = rep
15         self.sent = []
16
17     def sendto(self, data, addr):
18         self.sent.append((data, addr))
19
20     def recvfrom(self, n):
21         return self.rep.pop(0)
```

test_client.py

test_server.py

test_packat.py

Dans le fichier test_client.py, nous avons utilisé un FakeSocket pour simuler le réseau.



Plusieur cas sont tester comme :

url invalide	mauvais schéma HTTP	réception d'un fichier	envoi des Ack	écriture correcte du fichier
--------------	---------------------	------------------------	---------------	------------------------------

Dans test_server.py, nous avons tester :

- fichier inexistant
- envoi de donnée(DATA)
- recevoir et reconnaitre une requête GET
- répondre au bon client

Dans test_packet.py, nous avons tester:

- vérifier les champs après pack/unpack
- packeta avec payload vide
- type, window ou seqnum invalide
- payload trop grand
- packet trop court

```
----- coverage: platform linux, python 3.12.3-final-0 -----
Name          Stmts  Miss  Cover
-----
src/client.py   59      5   92%
src/packet.py   92     14   85%
src/server.py  100     20   80%
-----
TOTAL          251     39   84%
```

L'ensemble des tests effectués permet d'obtenir un coverage de 84%.

Tests d'interopérabilité

- Client.py fonctionne bien avec la plupart des autres groupes

Bugs probablement due au serveur de leur côté

- Server.py fonctionne avec 1 groupe mais n'as pas fonctionner avec 3 autre groupe

Problème sur le format du paquet avec un autre groupe.

Repérage d'un bug dans le code de notre serveur.

Bugs corrigés

- **Dans server.py, on utilisait packet.seqnum comme clé pour le dictionnaire timestamp lors de l'envoi, mais on utilisait i pour vérifier les timeouts.**

Le problème : Si le fichier fait plus de 2048 segments, le seqnum revient à 0, mais i continue de grimper. Le serveur ne retransmettra plus rien.

Correction : Utilise toujours l'index i comme clé.

- **On ajoute le paquet vide (EOF) à notre listes packets, et on fait un sock.sendto manuel juste après.**

Le problème : Le paquet de fin sera envoyé deux fois. Cela peut faire planter les clients des autres groupes.

Correction : Supprime le sock.sendto à l'intérieur de la boucle qui lit le fichier et laisser la fenêtre glissante gérer l'envoi de tous les paquets de la liste.

- **Ajout et correction de quelques tests**